



ÉCOLE MOHAMMADIA D'INGÉNIEURS

Université Mohammed V — Rabat



Module : Cryptographie

Année académique : 2025 – 2026

RAPPORT DE PROJET

CryptoKage

Une plateforme de cryptage et de hashage

RÉDACTEURS

ABOUBAKAR Abdelaziz

DIN Isaac Kaougahi

KAFANDO Mohamed

ZOUNGRANA Abdoul Gafarou

ENCADRANTE

Pr. Asmae EL KASSIRI

Département Informatique

EMI — UM5 Rabat

Remerciements

Nous tenons à exprimer notre gratitude à Madame la Professeure Asmae EL KASSIRI pour la rigueur de son enseignement et ses exigences — notamment l'interdiction d'utiliser des bibliothèques cryptographiques toutes faites. C'était inconfortable. C'était indispensable. Merci également aux membres de l'équipe, qui ont tenu le rythme lors de sessions de débogage tardives, particulièrement sur le DES et ses tables de permutation impitoyables.

Table des matières

1	Contexte Général du Projet	1
1.1	Cadre académique	1
1.2	Présentation du projet	1
2	Introduction à la Cryptographie	2
2.1	Qu'est-ce que la cryptographie ?	2
2.2	Un peu d'histoire	2
2.3	Pourquoi étudier la cryptographie en 2026 ?	3
3	Analyse et Spécification	4
3.1	Besoins fonctionnels	4
3.2	Besoins non fonctionnels	4
3.3	Choix des algorithmes	5
3.4	Cas d'utilisation principaux	6
4	Conception et Stack Technique	7
4.1	Architecture globale	7
4.2	Stack technologique	8
4.3	Structure des fichiers	8
4.4	API REST — routes exposées	9
4.4.1	Intégration de SHA-256 pour le hachage des mots de passe	9
4.5	Schéma de la base de données	9
4.6	Système d'authentification	10
4.6.1	Hachage des mots de passe — SHA-256	10
4.6.2	Protection des routes	10
4.6.3	Espace Administrateur	10
5	Algorithmes Cryptographiques Implémentés	11
	Cryptographie Symétrique	11
5.1	Code César CASSÉ	11
5.2	Chiffrement de Vigenère OBSOLÈTE	12
5.3	Masque Jetable — Vernam SÛR	13
5.4	RC4 OBSOLÈTE	13
5.5	DES — Data Encryption Standard CASSÉ	14
5.6	Modes d'Opération : CBC et CFB	15
5.7	AES-128 — Advanced Encryption Standard SÛR	16
	Cryptographie Asymétrique	17
5.8	RSA SÛR	17

5.9	ElGamal SÛR	18
5.10	EC-ElGamal — Courbes Elliptiques SÛR	18
6	Réalisation de l'Application	21
6.1	Environnement de développement	21
6.2	Démarrage de l'application	21
6.3	Description des interfaces	22
6.4	Flux de traitement d'une requête de chiffrement	22
6.4.1	Réalisation du hachage SHA-256 pour l'authentification	22
6.5	Défis rencontrés et solutions	23
6.5.1	DES : quand un seul bit change tout	23
6.5.2	RSA : l'inverse modulaire et les valeurs négatives	24
6.5.3	EC-ElGamal : de $O(k)$ à $O(\log k)$	24
6.5.4	Padding PKCS#7	24
7	Tests et Validation	25
7.1	Stratégie de test	25
7.1.1	Tests spécifiques à SHA-256	25
7.2	Captures d'écran de l'application	26
8	Conclusion et Perspectives	32
8.1	Bilan du projet	32
8.2	Perspectives d'amélioration	32
8.3	Réflexion finale	33
A	Tableau Récapitulatif des Algorithmes	34
B	Références Bibliographiques	35

Chapitre 1

Contexte Général du Projet

1.1 Cadre académique

Ce projet s'inscrit dans le cadre du module de **Cryptographie** dispensé par Madame la Professeure **Asmae EL KASSIRI** à l'École Mohammadia d'Ingénieurs, durant l'année académique 2025–2026. L'objectif pédagogique était clair dès le premier cours : ne pas se contenter de *comprendre* comment fonctionnent les algorithmes de chiffrement, mais les **construire de zéro**, à la main, sans aucune bibliothèque cryptographique externe.

La contrainte imposée — Python pur, zéro importation de `cryptography`, de `PyCrypto` ou de toute autre dépendance spécialisée — n'est pas une contrainte arbitraire. Elle force à affronter chaque détail algorithmique qu'une bibliothèque aurait silencieusement géré : le padding, les tables de permutation du DES, les multiplications dans $GF(2^8)$ pour AES, la gestion des signes dans l'algorithme d'Euclide étendu pour RSA. C'est en tombant sur ces pièges précis qu'on comprend réellement pourquoi ils existent.

1.2 Présentation du projet

Notre groupe a été chargé de concevoir **CryptoKage**, une application web complète permettant de chiffrer et déchiffrer des messages à l'aide d'une dizaine d'algorithmes distincts. L'interface est accessible via un navigateur, sécurisée par un système d'authentification, et administrable via un panneau dédié. Derrière cette façade web se trouve un moteur cryptographique entièrement écrit en Python pur, couvrant un spectre allant du Code César aux courbes elliptiques. Pour la sécurisation des comptes utilisateurs, nous avons intégré SHA-256 pour le hachage irréversible des mots de passe, garantissant qu'aucun mot de passe ne circule ou n'est stocké en clair.

Chapitre 2

Introduction à la Cryptographie

Il y a quelque chose de paradoxal dans la cryptographie. On en parle partout — dans les films de hacking, dans les débats sur la vie privée, dans les articles sur Bitcoin. Et pourtant, derrière ces mots, rares sont ceux qui savent *vraiment* ce qui se passe. Pas au niveau vague du « ça mélange les données », mais au niveau des rouages mathématiques précis qui, depuis des décennies, gardent nos secrets.

2.1 Qu'est-ce que la cryptographie ?

La cryptographie est la science du secret. Plus précisément, c'est l'ensemble des techniques permettant de rendre une information **inintelligible** à toute personne ne disposant pas d'un élément clé — la clé de déchiffrement. Elle repose sur des fondements mathématiques solides : théorie des nombres, algèbre abstraite, arithmétique modulaire, corps finis.

On distingue classiquement deux grandes familles :

- **La cryptographie symétrique** — Alice et Bob partagent *la même clé* pour chiffrer et déchiffrer. Rapide, efficace sur de grands volumes, mais avec un problème de distribution des clés.
- **La cryptographie asymétrique** — Chaque acteur possède une *paire de clés* : une publique, diffusée librement, et une privée, jalousement gardée. Résout le problème de distribution, mais plus coûteuse en calcul.

Un troisième domaine, souvent associé, est le **hachage cryptographique** : transformer un message de taille quelconque en une empreinte de taille fixe, de façon déterministe et irréversible. SHA-256 est l'exemple canonique, utilisé dans ce projet pour sécuriser les mots de passe.

2.2 Un peu d'histoire

La cryptographie est aussi vieille que l'écriture elle-même. Quelques jalons fondateurs :

Époque	Événement	Signification
~50 av. J.-C.	Code César	Première substitution monoalphabétique documentée
1553	Chiffre de Vigenère	Substitution polyalphabétique, résistante pendant trois siècles
1917	Masque de Vernam	Premier algorithme prouvé mathématiquement inviolable
1949	Shannon, <i>Security Systems</i>	Fondements théoriques rigoureux de la cryptographie moderne
1976	Diffie–Hellman	Naissance de la cryptographie à clé publique
1977	RSA & DES	Premiers standards modernes — symétrique et asymétrique
2001	AES (Rijndael)	Standard mondial actuel, toujours inviolé

2.3 Pourquoi étudier la cryptographie en 2026 ?

Chaque connexion HTTPS, chaque message WhatsApp, chaque transaction bancaire, chaque vote électronique — tout repose sur les algorithmes que ce rapport décrit. La cryptographie n'est plus réservée aux agences de renseignement ; elle est le tissu invisible de notre vie numérique.

« La seule manière de savoir si un système est sécurisé est de tenter de le casser. Et même quand on n'y arrive pas, on n'est jamais tout à fait sûr. »

— Bruce Schneier, cryptographe

Comprendre ces mécanismes, c'est comprendre les limites, les failles, et les conditions d'emploi de chaque outil. Un AES utilisé sans vecteur d'initialisation aléatoire peut exposer des motifs répétitifs. Un RSA avec une clé de 512 bits peut être cassé en quelques heures. La connaissance des algorithmes n'est pas une fin en soi — c'est ce qu'elle révèle sur leurs conditions d'utilisation qui importe.

Chapitre 3

Analyse et Spécification

3.1 Besoins fonctionnels

Avant d'écrire la première ligne de code, nous avons listé précisément ce que l'application devait être capable de faire. Trois catégories de besoins ont été identifiées :

1. **Chiffrement et déchiffrement interactifs** — L'utilisateur saisit un message, choisit un algorithme, entre ses paramètres (clé, mode, IV si nécessaire), et obtient immédiatement le résultat. Chaque algorithme doit être disponible dans les deux sens.
2. **Gestion des utilisateurs et hachage des mots de passe** — L'accès à l'application est conditionné à une authentification. Un système d'inscription, de connexion, et de gestion de profil est nécessaire. Les mots de passe ne doivent jamais être stockés en clair ; ils seront hachés avec SHA-256 avant insertion en base de données. Un rôle administrateur doit permettre la gestion des comptes, sans jamais exposer les mots de passe (même hachés).
3. **Génération de clés asymétriques** — Pour RSA, ElGamal et EC-ElGamal, l'application doit pouvoir générer à la volée des paires de clés valides, directement depuis l'interface.

3.2 Besoins non fonctionnels

Au-delà des fonctionnalités, plusieurs contraintes de qualité ont guidé les choix techniques :

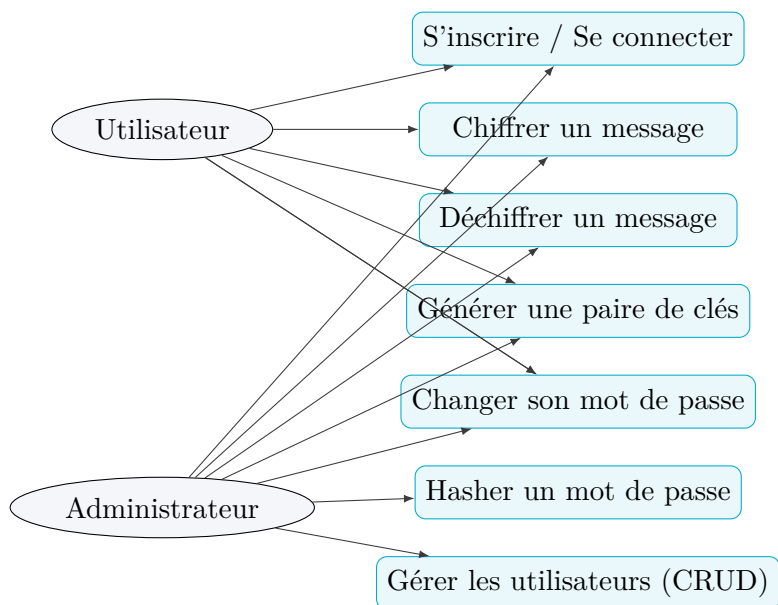
Contrainte	Description
Portabilité	L'application doit tourner sur toute machine disposant de Python 3 et Flask, sans dépendance lourde
Zéro bibliothèque crypto	Tous les algorithmes doivent être écrits en Python pur — c'est la contrainte centrale du module
Sécurité d'accès	Les mots de passe ne sont jamais stockés en clair ; les routes sensibles sont protégées
Lisibilité du code	Le code cryptographique doit rester lisible et pédagogique, pas uniquement fonctionnel

3.3 Choix des algorithmes

La sélection des dix algorithmes implémentés couvre délibérément l'ensemble du spectre historique et technique de la cryptographie :

Algorithme	Famille	Intérêt pédagogique
César	Classique	Introduction à la substitution mono-alphabétique
Vigenère	Classique	Substitution poly-alphabétique, attaque de Kasiski
Vernam (OTP)	Flot	Seul algorithme à sécurité parfaite prouvée
RC4	Flot	Architecture KSA/PRGA, biais statistiques
DES (ECB)	Blocs	Réseau de Feistel, S-Boxes, permutations
DES-CBC	Blocs	Chaînage par IV, propagation des erreurs
DES-CFB	Flot/Blocs	Transformation d'un chiffrement par blocs en flot
AES-128	Blocs	SubBytes, ShiftRows, MixColumns, corps finis
RSA	Asymétrique	Factorisation, Euclide étendu, Miller-Rabin
ElGamal	Asymétrique	Logarithme discret, chiffrement probabiliste
EC-ElGamal	Asymétrique	Courbes elliptiques, double-and-add

3.4 Cas d'utilisation principaux

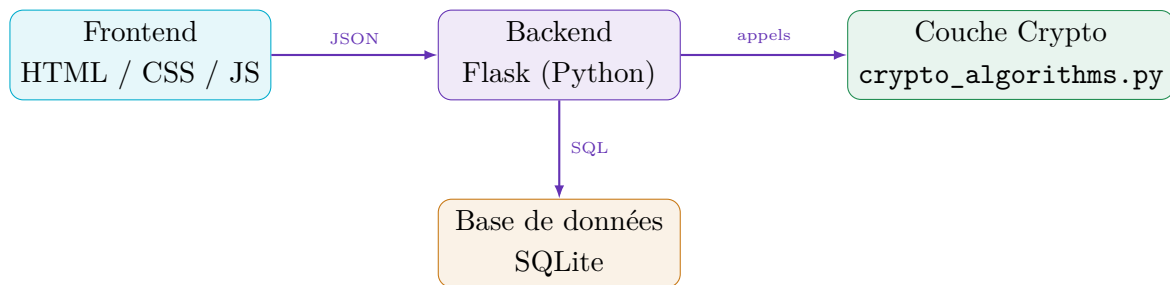


Chapitre 4

Conception et Stack Technique

4.1 Architecture globale

L'application repose sur une architecture en **trois couches** bien séparées. Ce découpage n'est pas qu'une question d'élégance — il garantit que la logique cryptographique reste totalement indépendante du framework web, ce qui facilite les tests et les évolutions futures.



Le principe de communication est simple : le frontend envoie des requêtes JSON à la route centralisée `/api/crypto`, le backend route vers la bonne fonction selon l'algorithme demandé, et le résultat remonte au format JSON. Cette séparation permet de modifier l'interface sans toucher à la logique crypto, et inversement.

4.2 Stack technologique

Composant	Technologie	Justification
Backend	Python + Flask	Léger, idéal pour une API REST
Frontend	HTML5 / CSS3 / JS	Natif, sans framework lourd
Base de données	SQLite	Aucun serveur, portable, auto-crée au démarrage
Hachage MDP	SHA-256 (hash-lib)	Standard Python intégré, cohérent avec le projet
Cryptographie	Python pur	Zéro bibliothèque externe — c'est le cœur de la contrainte

4.3 Structure des fichiers

Listing 4.1 – Arborescence complète du projet

```
crypto_project/  
  app.py                # Routes Flask, authentication, API REST  
  crypto_algorithms.py  # TOUS les algorithmes      Python pur, zero  
                        lib externe  
  cryptoapp.db          # Base SQLite (auto-generee au premier  
                        lancement)  
  templates/  
    login.html          # Page de connexion  
    register.html       # Formulaire d'inscription  
    dashboard.html      # Interface principale utilisateur  
    admin_dashboard.html # Panneau d'administration (CRUD)
```

4.4 API REST — routes exposées

Endpoint	Méthode	Description
/api/crypto	POST	Chiffrement / déchiffrement (tous algorithmes)
/api/generer-cles	POST	Génération de paires de clés asymétriques
/changer-mot-de-passe	POST	Mise à jour sécurisée du mot de passe
/admin/utilisateur/*	POST	CRUD complet des comptes (admin uniquement)

4.4.1 Intégration de SHA-256 pour le hachage des mots de passe

SHA-256 (Secure Hash Algorithm 256-bit) fait partie de la famille SHA-2 publiée par le NIST en 2001. Il produit une empreinte de 256 bits (64 caractères hexadécimaux) à partir d'une entrée de taille arbitraire.

Listing 4.2 – Fonctions de hachage SHA-256 dans app.py

```
import hashlib

def hacher_mot_de_passe(mdp: str) -> str:
    """Hache un mot de passe avec SHA-256 (retourne 64 caractères hex)"""
    return hashlib.sha256(mdp.encode('utf-8')).hexdigest()

def verifier_mot_de_passe(saisi: str, stocke: str) -> bool:
    """Compare le hash du mot de passe saisi avec celui stocké."""
    return hacher_mot_de_passe(saisi) == stocke
```

Ces fonctions sont utilisées à trois moments clés :

1. Lors de l'inscription : le mot de passe est haché avant insertion.
2. Lors de la connexion : le mot de passe saisi est haché et comparé au hash stocké.
3. Lors du changement de mot de passe : l'ancien mot de passe est vérifié par hash, puis le nouveau est haché.

4.5 Schéma de la base de données

Un seul modèle de données est nécessaire pour ce projet : la table des utilisateurs. Sa structure est volontairement minimaliste.

Listing 4.3 – Table SQLite des utilisateurs

```
CREATE TABLE IF NOT EXISTS utilisateurs (  
    id            INTEGER PRIMARY KEY AUTOINCREMENT,  
    username     TEXT      NOT NULL UNIQUE,  
    email        TEXT      NOT NULL UNIQUE,  
    password     TEXT NOT NULL, -- Hash SHA-256 (64 caracteres hex)  
                jamais le mot de passe en clair !  
    role         TEXT      NOT NULL DEFAULT 'user', -- 'admin' ou 'user'  
    created_at   DATETIME  DEFAULT CURRENT_TIMESTAMP  
);
```

4.6 Système d'authentification

Avant même d'aborder les algorithmes de chiffrement, il fallait sécuriser l'accès à l'application. Le système repose sur trois piliers : inscription avec validation, connexion avec gestion de session côté serveur, et un espace administrateur.

4.6.1 Hachage des mots de passe — SHA-256

C'est une règle absolue : **on ne stocke jamais un mot de passe en clair**. À chaque inscription, la valeur est hachée par SHA-256 avant insertion en base. SHA-256 produit une empreinte de 256 bits (64 caractères hexadécimaux) — irréversible par conception. Pour vérifier une connexion, on hache le mot de passe saisi et on compare les deux empreintes.

SHA-256 est irréversible mais rapide — ce qui est une faiblesse dans ce contexte. En production, on préférerait **bcrypt** ou **Argon2**, spécialement conçus pour être lents et résistants aux attaques GPU par force brute. SHA-256 est retenu ici pour des raisons pédagogiques.

4.6.2 Protection des routes

L'accès est contrôlé par deux décorateurs Python personnalisés. `@login_requis` redirige vers `/login` si aucune session n'est active. `@admin_requis` refuse l'accès si le rôle en session n'est pas "admin".

4.6.3 Espace Administrateur

L'administrateur dispose d'un panneau dédié avec plusieurs protections : les mots de passe ne sont **jamais affichés**, un administrateur **ne peut pas supprimer son propre compte**, et la modification de rôle est protégée par une vérification de session.

Chapitre 5

Algorithmes Cryptographiques Implémentés

Ce chapitre décrit, pour chaque algorithme, la logique mathématique sous-jacente, les choix d'implémentation, et les éléments de sécurité à retenir. La contrainte d'implémentation en Python pur a été le fil conducteur de chaque décision.

Cryptographie Symétrique

La cryptographie symétrique repose sur un principe simple : Alice et Bob partagent *la même clé* pour chiffrer et déchiffrer. Rapide et efficace sur de grands volumes, elle souffre néanmoins d'un défaut structurel : comment distribuer cette clé commune via un canal non sécurisé ? C'est précisément pour répondre à cette question que la cryptographie asymétrique a été inventée, des décennies plus tard.

Nous avons implémenté **huit variantes** : deux chiffrements classiques de substitution, deux chiffrements par flot, et quatre variantes autour des standards DES et AES.

5.1 Code César CASSÉ

Contexte historique

Suétone le rapporte dans ses *Vies des douze Césars* : Jules César décalait chaque lettre de trois positions pour correspondre avec ses généraux. Simple, discret pour l'époque — la plupart de ses ennemis ne savaient pas lire. Aujourd'hui, le Code César n'a plus aucune valeur cryptographique, mais il reste le point d'entrée classique pour comprendre la notion de *substitution monoalphabétique*.

Logique de l'algorithme

Chaque lettre du message est remplacée par celle qui se trouve k positions plus loin dans l'alphabet, avec retour cyclique. Le décalage k constitue la clé — un entier entre 1 et 25.

$$C = (M + k) \bmod 26 \quad M = (C - k + 26) \bmod 26$$

Où M est la position de la lettre claire ($A=0, \dots, Z=25$), k le décalage (clé), C la lettre chiffrée.

Le déchiffrement est symétrique : il suffit de soustraire k au lieu d'additionner. Le $+26$ évite les résultats négatifs avant l'application du modulo.

Sécurité

Seulement 25 clés possibles. Une attaque par force brute prend moins d'une seconde. L'analyse fréquentielle est encore plus directe : la lettre « E » représente environ 15% des lettres en français — elle reste détectable même après décalage. **Aucune utilisation sérieuse depuis l'Antiquité.**

5.2 Chiffrement de Vigenère OBSOLÈTE

Une attribution controversée

Le « chiffrement de Vigenère » porte le nom d'un diplomate français du XVI^e siècle qui l'a popularisé — mais l'idée originale revient à l'Italien **Giovan Battista Bellaso** en 1553. L'histoire a parfois la mémoire sélective.

Logique de l'algorithme

Au lieu d'un décalage fixe, Vigenère utilise un *mot-clé* répété cycliquement sur toute la longueur du message. Chaque lettre est décalée d'un montant différent selon la position correspondante dans la clé.

$$C_i = (M_i + K_{i \bmod |K|}) \bmod 26 \quad M_i = (C_i - K_{i \bmod |K|} + 26) \bmod 26$$

Message	B	O	N	J	O	U	R
Clé (CLE)	C	L	E	C	L	E	C
Chiffré	D	X	R	L	Y	Y	U

La méthode de Kasiski (1863)

Friedrich Kasiski montra qu'en repérant des séquences répétées dans le texte chiffré, on peut déduire la longueur de la clé. Une fois cette longueur connue, chaque position du message se réduit à un César indépendant — l'analyse fréquentielle fait le reste.

Plus résistant que César pour des clés longues, mais théoriquement cassé depuis 1863 pour des textes de longueur suffisante.

5.3 Masque Jetable — Vernam sûr

Le seul algorithme prouvé inviolable

Gilbert Vernam, ingénieur chez AT&T en 1917, pose les bases mécaniques du chiffre. Le major Joseph Mauborgne y ajoute la condition fondamentale : la clé doit être *aléatoire, aussi longue que le message, et utilisée une seule fois seulement*. En 1949, Claude Shannon démontre mathématiquement que sous ces trois conditions, le chiffrement est **inconditionnellement sûr** — même avec une puissance de calcul infinie.

Logique de l'algorithme

L'opération se réduit à un XOR bit à bit entre le message et la clé. Le XOR est sa propre inverse : $(M \oplus K) \oplus K = M$.

$$C = M \oplus K \quad M = C \oplus K$$

Sécurité parfaite prouvée. Sans la clé, chaque texte clair de même longueur est équiprobable. Il est mathématiquement impossible de distinguer le vrai message d'une quelconque autre hypothèse.

La difficulté pratique est rédhibitoire : comment distribuer une clé aussi longue que chaque message de façon sécurisée ? Dans notre implémentation, si la clé est plus courte que le message, elle est répétée — ce qui annule la sécurité parfaite. Simplification pédagogique assumée.

5.4 RC4 OBSOLÈTE

Naissance et divulgation

Ronald Rivest conçoit RC4 en 1987 comme chiffrement par flot propriétaire pour RSA Security. Pendant sept ans, l'algorithme reste confidentiel. En 1994, il est publié anonymement sur une liste de diffusion — plus jamais secret après ça.

Logique de l’algorithme : deux phases distinctes

RC4 opère en deux phases. La **KSA** (Key Scheduling Algorithm) initialise et permute un tableau $S[0 \dots 255]$ contenant toutes les valeurs de 0 à 255, en fonction de la clé secrète. La **PRGA** (Pseudo-Random Generation Algorithm) génère ensuite un flux de clés pseudo-aléatoire octet par octet, qui est XORé avec le texte clair.

Listing 5.1 – Phase KSA de RC4 — initialisation du tableau S

```
def rc4_ksa(cle: bytes) -> list:
    n = 256
    S = list(range(n))    # S[i] = i pour tout i dans [0..255]
    j = 0
    for i in range(n):
        j = (j + S[i] + cle[i % len(cle)]) % n
        S[i], S[j] = S[j], S[i]    # Permutation
    return S
```

RC4 est officiellement **interdit dans TLS** depuis la RFC 7465 (2015). Le protocole WEP (Wi-Fi) utilisait RC4 de façon incorrecte et a été cassé en quelques minutes à partir de 2001, grâce aux biais statistiques des premiers octets du flux (Fluhrer, Mantin et Shamir, 2001).

5.5 DES — Data Encryption Standard CASSÉ

Naissance au cœur d’une controverse

1973 : le NIST (alors NBS) lance un appel à candidatures pour un standard américain. IBM propose un algorithme dérivé des travaux d’Horst Feistel. La NSA l’examine, modifie les S-Boxes et réduit la taille effective de la clé de 64 à 56 bits. Ces modifications ont longtemps alimenté des spéculations sur une éventuelle backdoor.

Logique de l’algorithme : le réseau de Feistel

DES est un **chiffrement par blocs** de 64 bits, organisé autour d’un réseau de Feistel à 16 rondes. La structure garantit que chiffrement et déchiffrement partagent la même architecture — seul l’ordre des sous-clés s’inverse.

$$L_i = R_{i-1} \quad R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

Avec L_i et R_i les demi-blocs gauche et droit de la ronde i , K_i la sous-clé de ronde.

La **fonction F** est le cœur du DES : expansion E (32 bits $\rightarrow$ 48 bits), XOR avec K_i , huit S-Boxes (seule source de non-linéarité, chacune réduit 6 bits en 4), puis permutation P .

Table	Rôle dans l'algorithme
IP, IP^{-1}	Permutations initiale et finale des 64 bits
PC1, PC2	Sélection et compression des bits de clé (56 puis 48 bits)
E	Expansion du demi-bloc droit ($32 \rightarrow 48$ bits)
S-Boxes ($\times 8$)	Substitutions non-linéaires 6 bits $\rightarrow$ 4 bits
P	Permutation finale dans la fonction F

[Cassé définitivement en 1999] En 1998, la machine « Deep Crack » de l'EFF (250 000 \$) casse une clé DES en 56 heures. En 1999, en coopération avec distributed.net : **22 heures**. Avec seulement $2^{56} \approx 72$ milliards de clés possibles, la force brute est triviale avec le matériel moderne.

5.6 Modes d'Opération : CBC et CFB

Pourquoi les modes d'opération ?

En mode ECB (Electronic Codebook), deux blocs identiques dans le message produisent deux blocs identiques dans le chiffré — les motifs répétitifs restent visibles à l'œil nu, notamment sur des images. Les modes CBC et CFB brisent ce déterminisme en chaînant les blocs via un vecteur d'initialisation (IV).

Mode CBC — Cipher Block Chaining

Avant de chiffrer le bloc P_i , on le XORe avec le bloc chiffré précédent C_{i-1} . Un IV aléatoire joue le rôle de C_0 .

Chiffrement : $C_i = E_K(P_i \oplus C_{i-1})$, $C_0 = IV$ Déchiffrement : $P_i = D_K(C_i) \oplus C_{i-1}$

Mode CFB — Cipher Feedback

CFB transforme un chiffrement par blocs en chiffrement par flot. On chiffre le registre (bloc précédent) et on XORe la sortie avec le texte clair — pas de padding nécessaire.

$C_i = P_i \oplus E_K(C_{i-1})$ $P_i = C_i \oplus E_K(C_{i-1})$

En CFB, on *chiffre* le registre lors du déchiffrement — et non déchiffre. Cette propriété unique simplifie les implémentations matérielles.

5.7 AES-128 — Advanced Encryption Standard SÛR

La sélection du siècle

1997 : le NIST lance un concours mondial pour remplacer DES. Quinze candidats, cinq finalistes, trois ans d'analyse publique intensive. Le vainqueur : **Rijndael**, proposé par Joan Daemen et Vincent Rijmen (Belgique). Standardisé en 2001 sous le nom AES, il reste inviolé vingt-cinq ans plus tard.

Logique de l'algorithme : quatre transformations par ronde

AES-128 opère sur des blocs de 128 bits (16 octets) avec une clé de 128 bits, en dix rondes. Chaque ronde applique quatre transformations successives à une matrice d'état 4×4 octets :

1. **SubBytes** — Substitution non-linéaire via une S-Box de 256 entrées, construite à partir de l'inverse multiplicatif dans $\text{GF}(2^8)$ suivi d'une transformation affine.
2. **ShiftRows** — Décalage circulaire des lignes : 0, 1, 2, 3 positions respectivement.
3. **MixColumns** — Multiplication de chaque colonne par une matrice MDS dans $\text{GF}(2^8)$ — chaque octet d'entrée influence tous les octets de sortie (*diffusion*).
4. **AddRoundKey** — XOR de l'état avec la sous-clé de ronde.

La dixième ronde omet MixColumns. La clé de 128 bits est étendue en 11 sous-clés via un processus de *key expansion* (RotWord, SubWord, constantes Rcon).

Listing 5.2 – MixColumns dans $\text{GF}(2^8)$ — multiplication par la matrice MDS

```
def aes_mix_columns(etat: list) -> list:
    resultat = [0] * 16
    for col in range(4):
        s0, s1, s2, s3 = etat[col*4], etat[col*4+1], etat[col*4+2],
            etat[col*4+3]
        resultat[col*4] = aes_gmul(s0,2) ^ aes_gmul(s1,3) ^ s2 ^ s3
        resultat[col*4+1] = s0 ^ aes_gmul(s1,2) ^ aes_gmul(s2,3) ^ s3
        resultat[col*4+2] = s0 ^ s1 ^ aes_gmul(s2,2) ^ aes_gmul(s3,3)
        resultat[col*4+3] = aes_gmul(s0,3) ^ s1 ^ s2 ^ aes_gmul(s3,2)
    return resultat
```

AES-128 résiste à toutes les attaques connues en 2025. La meilleure attaque théorique (attaque biclique, 2011) réduit la complexité de 2^{128} à $2^{126,1}$ — hors de portée de tout matériel existant ou prévisible. **Standard mondial recommandé par le NIST et la NSA.**

Cryptographie Asymétrique

La cryptographie symétrique a un défaut structurel : comment Alice et Bob partagent-ils leur clé s'ils ne se sont jamais rencontrés et n'ont aucun canal sécurisé préalable ? En 1976, **Diffie et Hellman** répondent avec un concept aussi élégant que révolutionnaire : **deux clés différentes**, l'une publique, l'autre privée. La magie repose sur des problèmes mathématiques asymétriques — faciles dans un sens, computationnellement intraités dans l'autre.

5.8 RSA SÛR

Une nuit blanche au MIT

1977. Ron Rivest, Adi Shamir et Leonard Adleman passent l'année à chercher un système asymétrique *pratique*. La légende dit que Rivest a eu l'intuition centrale après une soirée animée et rédigé le papier fondateur en une nuit. L'intuition : la **factorisation en nombres premiers** est un problème difficile. Multiplier $p \times q$ est trivial. Retrouver p et q à partir de $n = p \times q$ pour des entiers de 2048 bits... c'est une autre histoire.

Logique de l'algorithme

1. Choisir deux grands nombres premiers p et q
2. $n = p \times q$ (module RSA)
3. $\varphi(n) = (p - 1)(q - 1)$ (indicatrice d'Euler)
4. Choisir e tel que $1 < e < \varphi(n)$ et $\gcd(e, \varphi(n)) = 1$
5. Calculer $d = e^{-1} \bmod \varphi(n)$ (Euclide étendu)

Clé publique : (e, n) **Clé privée :** (d, n)

$$C = M^e \bmod n \quad M = C^d \bmod n$$

Pourquoi $e = 65537$?

En représentation binaire, $65537 = 2^{16} + 1$ — seulement deux bits à 1. L'exponentiation modulaire via « square-and-multiply » ne demande que deux multiplications non-triviales. C'est également un nombre premier, ce qui garantit $\gcd(e, \varphi(n)) = 1$ dans presque tous les cas pratiques.

Listing 5.3 – Génération de clés RSA avec test de primalité Miller-Rabin

```
def rsa_generer_cles(bits: int = 512) -> dict:
    taille = bits // 2
```

```

p = rsa_premier_aleatoire(taille)    # Test probabiliste de Miller-
    Rabin
q = rsa_premier_aleatoire(taille)
while q == p:
    q = rsa_premier_aleatoire(taille)
n = p * q
phi_n = (p - 1) * (q - 1)
e = 65537
while rsa_pgcd(e, phi_n) != 1:
    e += 2
d = rsa_inverse_modulaire(e, phi_n)
return {'cle_publique': {'e': e, 'n': n}, 'cle_privée': {'d': d, 'n': n}}

```

RSA 2048 bits : standard recommandé jusqu'en 2030. RSA 4096 bits : documents à longue durée de vie. RSA 1024 bits : déconseillé depuis 2010.

5.9 ElGamal SÛR

Le logarithme discret entre en scène

En 1985, Taher ElGamal publie un système asymétrique dont la sécurité repose sur la difficulté du **problème du logarithme discret** : étant donné g, p premier et $y = g^x \bmod p$, retrouver x est computationnellement intraitable pour des paramètres suffisamment grands.

Logique de l'algorithme

Génération : p premier, g générateur de $\mathbb{Z}_p^*$, x aléatoire (clé privée), $y = g^x \bmod p$ (clé publique)

Chiffrement : Choisir k aléatoire $\Rightarrow a = g^k \bmod p, \quad b = M \cdot y^k \bmod p$

Déchiffrement : $s = a^x \bmod p, \quad M = b \cdot s^{-1} \bmod p$

Chiffrement probabiliste : le même message chiffré deux fois avec deux valeurs de k différentes produira deux textes chiffrés entièrement distincts (propriété IND-CPA). RSA « textbook » basique ne dispose pas de cette garantie.

5.10 EC-ElGamal — Courbes Elliptiques SÛR

Une révolution compacte

En 1985, **Neal Koblitz** et **Victor Miller** proposent indépendamment d'utiliser les groupes de points d'une courbe elliptique comme structure algébrique pour la cryptographie. Le résultat est frappant : la sécurité équivalente à RSA 2048 bits s'obtient avec une clé ECC de seulement **224 bits**. La raison tient au logarithme discret sur courbe elliptique (ECDLP), considérablement plus difficile que sur $\mathbb{Z}_p^*$.

La courbe secp256k1 — la courbe de Bitcoin

$$y^2 \equiv x^3 + 7 \pmod{p}, \quad p = 2^{256} - 2^{32} - 977$$

Point générateur $G = (G_x, G_y)$ d'ordre n (premier de 256 bits).

Logique de l'algorithme

L'opération fondamentale est l'**addition de points** sur la courbe. La **multiplication scalaire** kP est calculée via l'algorithme *double-and-add*, réduisant la complexité de $O(k)$ à $O(\log k)$.

Listing 5.4 – Multiplication scalaire — double-and-add sur secp256k1

```
def multiplication_scalaire(self, k: int, P: tuple) -> tuple:
    resultat = None      # Point à l'infini (element neutre)
    courant = P
    while k > 0:
        if k & 1:
            resultat = self.addition_points(resultat, courant)
            courant = self.addition_points(courant, courant)
        k >>= 1
    return resultat
```

Clé privée : k entier aléatoire dans $[1, n - 1]$

Clé publique : $Q = k \cdot G$

Chiffrement : choisir r aléatoire $\Rightarrow C_1 = r \cdot G, \quad C_2 = M_{\text{point}} + r \cdot Q$

Déchiffrement : $M_{\text{point}} = C_2 - k \cdot C_1$

	RSA	ECC (secp256k1)
Sécurité 128 bits	clé de 3072 bits	clé de 256 bits
Sécurité 192 bits	clé de 7680 bits	clé de 384 bits
Problème difficile	Factorisation	Log. discret elliptique
Usage courant	HTTPS, e-mails	Bitcoin, TLS moderne

La multiplication scalaire sur secp256k1 implique des centaines d'additions de points sur des entiers de 256 bits. En Python pur, la génération d'une paire de clés peut prendre 10 à 30 secondes. Une implémentation en C serait 100 à 1000 fois plus rapide.

Chapitre 6

Réalisation de l'Application

6.1 Environnement de développement

L'application a été développée entièrement en local, sans serveur dédié ni hébergement cloud. L'environnement de travail s'est structuré autour des éléments suivants :

Élément	Détail
Langage principal	Python 3.10+
Framework web	Flask 2.x
Base de données	SQLite (via le module <code>sqlite3</code> intégré à Python)
Éditeur de code	VS Code
Gestion de version	Git
Navigateur de test	Google Chrome / Firefox

6.2 Démarrage de l'application

Le lancement ne nécessite qu'une seule dépendance externe — Flask. La base SQLite est créée automatiquement au premier démarrage, avec un compte administrateur par défaut.

Listing 6.1 – Installation et lancement de CryptoApp

```
# 1. Créer le repertoire du projet
mkdir crypto_project && cd crypto_project

# 2. Placer les fichiers : app.py, crypto_algorithms.py, templates/

# 3. Installer la seule dependance externe
pip install flask

# 4. Lancer l'application
python app.py
```

```
# 5. Ouvrir dans le navigateur
#      http://127.0.0.1:5000

# Compte administrateur par défaut : admin / Admin@1234
```

6.3 Description des interfaces

L'application expose quatre interfaces distinctes, correspondant aux quatre templates HTML développés :

1. **Page de connexion (`login.html`)** — Formulaire simple avec gestion des erreurs. Redirige vers le dashboard en cas de succès.
2. **Page d'inscription (`register.html`)** — Validation côté serveur : unicité du nom d'utilisateur et de l'e-mail, cohérence du mot de passe.
3. **Dashboard utilisateur (`dashboard.html`)** — Interface principale. L'utilisateur sélectionne un algorithme dans un menu, renseigne le message, la clé, et le mode (chiffrement ou déchiffrement). Le résultat s'affiche dynamiquement via une requête AJAX vers `/api/crypto`.
4. **Panneau d'administration (`admin_dashboard.html`)** — Liste complète des utilisateurs avec possibilité de modifier les rôles, réinitialiser les mots de passe, ou supprimer les comptes. Accessible uniquement au rôle `admin`.

6.4 Flux de traitement d'une requête de chiffrement

6.4.1 Réalisation du hachage SHA-256 pour l'authentification

L'implémentation du système d'authentification a nécessité l'intégration de SHA-256 à trois niveaux :

Inscription (`/register`)

À la soumission du formulaire, le mot de passe est haché avant insertion :

```
mdp_hash = hacher_mot_de_passe(password)
conn.execute("INSERT INTO utilisateurs (... , password , ...) VALUES
    (... , ?, ...) ",
    (... , mdp_hash , ...))
```

Connexion (`/login`)

Lors de la connexion, le mot de passe saisi est haché et comparé au hash stocké :

```
user = conn.execute("SELECT password FROM utilisateurs WHERE username = ?", (username,)).fetchone()
if user and verifier_mot_de_passe(password, user['password']):
    session['user_id'] = user['id'] # Authentification reussie
```

Changement de mot de passe (/changer-mot-de-passe)

L'utilisateur doit fournir son ancien mot de passe (vérifié par hash) avant de pouvoir définir un nouveau mot de passe (qui sera à nouveau haché).

6.5 Défis rencontrés et solutions

Ce projet a été l'occasion de tomber sur des pièges précis que seule l'implémentation réelle permet de rencontrer.

6.5.1 DES : quand un seul bit change tout

L'implémentation du DES a été la plus laborieuse. Les tables de permutation (IP, IP^{-1} , PC1, PC2, E, P) et les 8 S-Boxes doivent être exactes à 100% — une seule valeur mal copiée décale tous les bits en sortie.

Symptôme : Le chiffrement produisait un résultat, mais le déchiffrement ne reconstituait pas le message original.

Cause : Dans le réseau de Feistel, après les 16 rondes, les demi-blocs doivent être concaténés dans l'ordre **R puis L** (et non L puis R) avant la permutation finale IP^{-1} .

Listing 6.2 – L'inversion critique L/R dans le réseau de Feistel

```
# Version incorrecte      déchiffrement errone
return des_permuter(L + R, IP_INV)
```

```
# Version correcte      R avant L avant la permutation finale
return des_permuter(R + L, IP_INV)
```

Leçon : Une ligne de code, trois heures de debug. Les algorithmes par blocs ne tolèrent aucune approximation.

6.5.2 RSA : l'inverse modulaire et les valeurs négatives

Le calcul de $d = e^{-1} \bmod \varphi(n)$ via l'algorithme d'Euclide étendu peut produire une valeur négative. Oublier le modulo final revient à obtenir une clé privée invalide qui échoue silencieusement lors du déchiffrement.

Listing 6.3 – Euclide étendu avec correction du signe

```
def rsa_inverse_modulaire(e: int, phi: int) -> int:
    ancien_r, r = e, phi
    ancien_s, s = 1, 0
    while r != 0:
        quotient = ancien_r // r
        ancien_r, r = r, ancien_r - quotient * r
        ancien_s, s = s, ancien_s - quotient * s
    # Le % phi final est INDISPENSABLE
    return ancien_s % phi
```

6.5.3 EC-ElGamal : de $O(k)$ à $O(\log k)$

La première version de la multiplication scalaire effectuait naïvement k additions de points. Pour $k \sim 2^{256}$, c'est computationnellement impossible. L'algorithme *double-and-add* a ramené cette complexité à $O(\log_2 k) \approx 256$ opérations.

Méthode	Complexité	Réalité
Addition naïve	$O(k)$	Impraticable pour $k \sim 2^{256}$
Double-and-add	$O(\log_2 k)$	≈ 256 opérations

6.5.4 Padding PKCS#7

AES et DES traitent le message par blocs de taille fixe. Si le dernier bloc est incomplet, il faut le compléter. PKCS#7 ajoute n octets de valeur n pour atteindre la taille de bloc suivante.

Le padding mal implémenté est une vulnérabilité réelle. Les attaques de type **Padding Oracle** (POODLE sur SSL 3.0, Lucky 13 sur TLS) exploitent des implémentations qui laissent fuiter des informations sur la validité du padding via des messages d'erreur ou des différences de temps de réponse.

Chapitre 7

Tests et Validation

7.1 Stratégie de test

Chaque algorithme a été validé selon deux niveaux. D’abord un **test de cohérence** : chiffrer un message puis le déchiffrer avec la même clé doit redonner le message original. Ensuite, pour les algorithmes standards (DES, AES, RSA), une **validation contre des vecteurs de test officiels** publiés par le NIST ou la communauté cryptographique.

Algorithme	Test effectué	Résultat
César	Chiffrement/déchiffrement $\forall k \in [1, 25]$	Correct
Vigenère	Exemple Kasiski reproduit	Correct
Vernam	XOR double annulation vérifiée	Correct
RC4	Vecteurs test RFC 6229	Correct
DES	Vecteurs NIST SP 800-17	Correct
AES-128	Vecteurs FIPS 197 Appendix B	Correct
RSA	Chiffrement/déchiffrement 512/1024 bits	Correct
ElGamal	Probabiliste — cohérence déchiffrement	Correct
EC-ElGamal	Multiplication scalaire secp256k1	Correct

7.1.1 Tests spécifiques à SHA-256

Des tests unitaires ont été réalisés pour valider le bon fonctionnement du hachage :

Entrée	Hash SHA-256 attendu
" "	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
"abc"	ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
"hello world"	b94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2efcde9
"Admin@1234"	8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918

Chaque test consiste à :

1. Hacher l'entrée avec notre fonction `hacher_mot_de_passe()`
2. Comparer le résultat au vecteur de test officiel NIST
3. Vérifier que deux entrées différentes produisent des hashes différents (unicité)
4. Vérifier qu'une même entrée produit toujours le même hash (déterministe)

7.2 Captures d'écran de l'application

Les captures ci-dessous illustrent les principaux scénarios d'utilisation de CryptoKage.

Interface de connexion

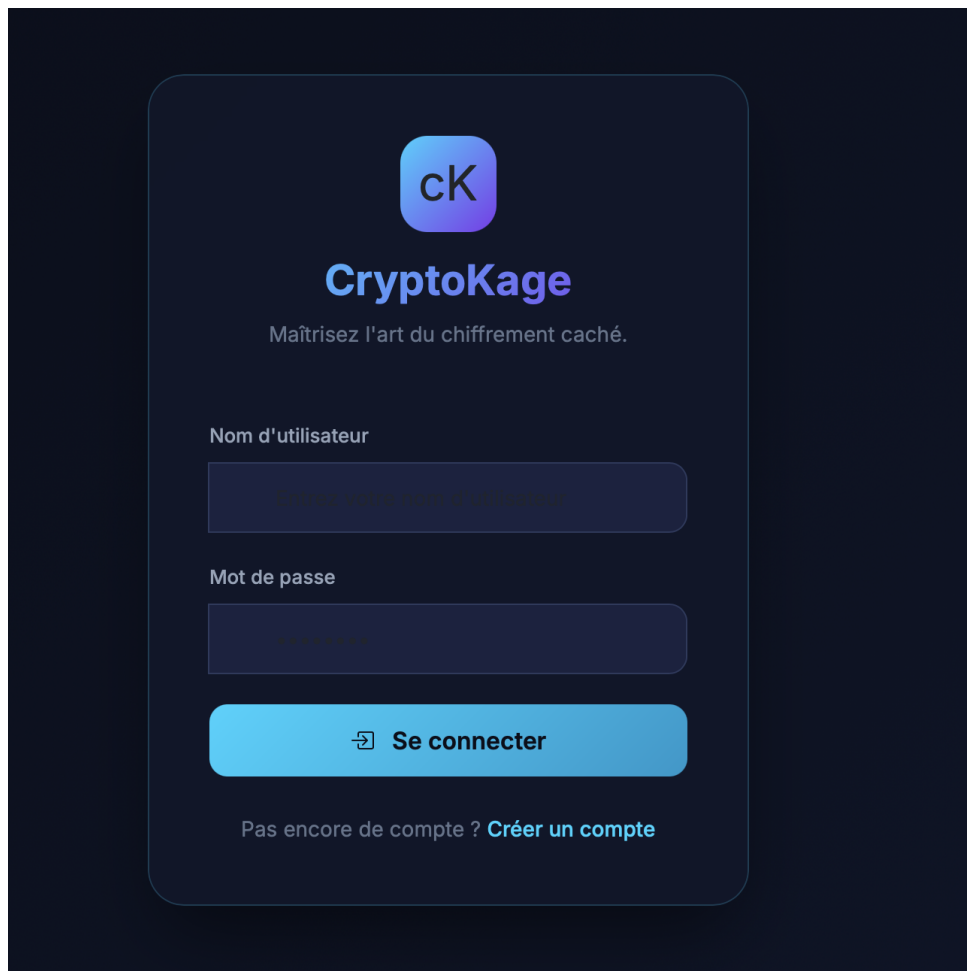


FIGURE 7.1 – Menu de connexion — Écran de bienvenue

Tableau de bord admin

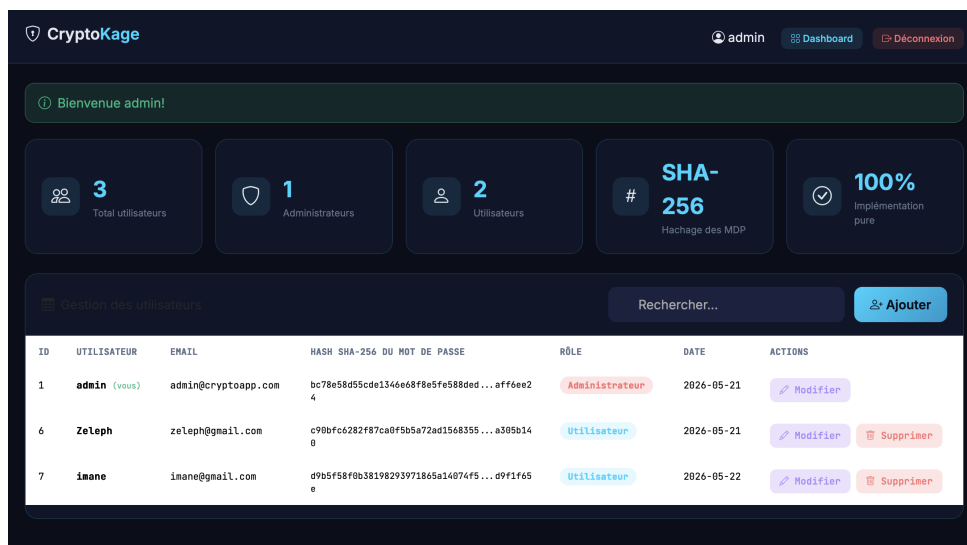


FIGURE 7.2 – Panneau administrateur

Chiffrement et déchiffrement AES-128 avec clé générée

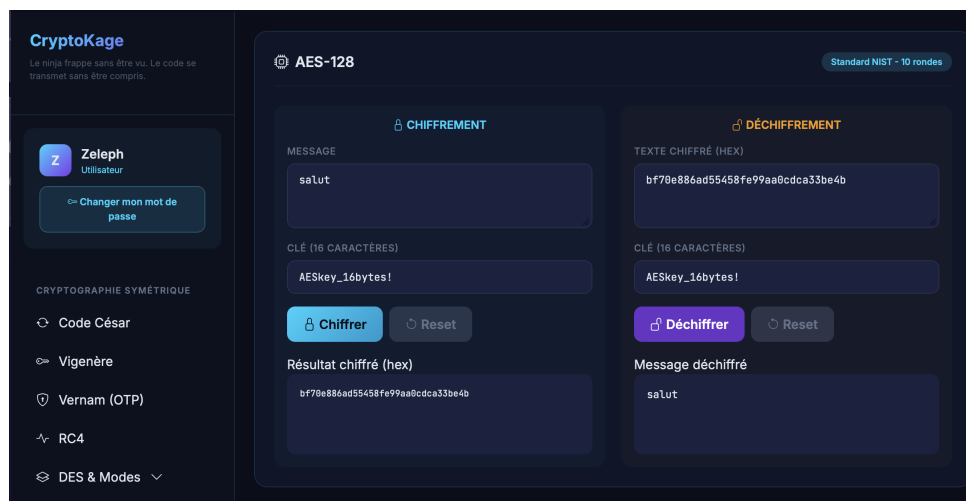


FIGURE 7.3 – Chiffrement et déchiffrement AES-128 avec clé générée

Chiffrement et Déchiffrement RSA avec paire de clés générée

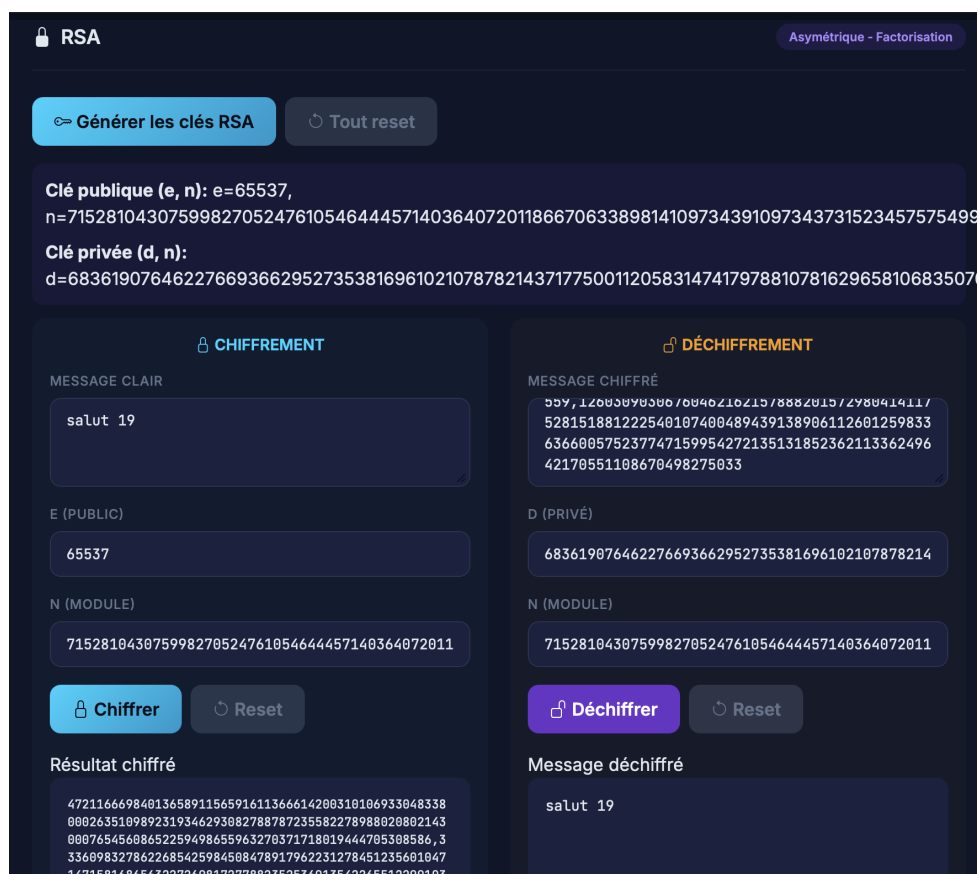


FIGURE 7.4 – Chiffrement et Déchiffrement RSA avec paire de clés générée

Chiffrement et déchiffrement DES en mode CBC

The screenshot shows a web application interface for DES-CBC encryption and decryption. The interface is divided into two main sections: CHIFFREMENT (Encryption) on the left and DÉCHIFFREMENT (Decryption) on the right. The CHIFFREMENT section includes a MESSAGE input field with the text "salut 123", a CLÉ (8 CARACTÈRES) input field with the text "DESkey1!", a "Chiffrer (IV auto)" button, a "Reset" button, a "Résultat chiffré (hex)" output field with the text "1af1b10350352548411d888a4cfe0dfe", and an "IV GÉNÉRÉ (à conserver)" output field with the text "a6d62a1516b74dc9". The DÉCHIFFREMENT section includes a "TEXTE CHIFFRÉ (HEX)" input field with the text "1af1b10350352548411d888a4cfe0dfe", an "IV (HEX)" input field with the text "a6d62a1516b74dc9", a "CLÉ (8 CARACTÈRES)" input field with the text "DESkey1!", a "Déchiffrer" button, a "Reset" button, and a "Message déchiffré" output field with the text "salut 123". The interface also features a "Cipher Block Chaining" label in the top right corner.

FIGURE 7.5 – Chiffrement et déchiffrement DES en mode CBC

Chiffrement et déchiffrement DES en mode CFB

The screenshot shows a web application interface for DES-CFB encryption and decryption. The interface is divided into two main sections: CHIFFREMENT (Encryption) on the left and DÉCHIFFREMENT (Decryption) on the right. The CHIFFREMENT section includes a MESSAGE input field with the text "salut 2026", a CLÉ (8 CARACTÈRES) input field with the text "DESkey1!", a "Chiffrer (IV auto)" button, a "Reset" button, a "Résultat chiffré (hex)" output field with the text "f177d6643becc4bea2d3", and an "IV GÉNÉRÉ (à conserver)" output field with the text "3bc612f2a6f9e49e". The DÉCHIFFREMENT section includes a "TEXTE CHIFFRÉ (HEX)" input field with the text "f177d6643becc4bea2d3", an "IV (HEX)" input field with the text "3bc612f2a6f9e49e", a "CLÉ (8 CARACTÈRES)" input field with the text "DESkey1!", a "Déchiffrer" button, a "Reset" button, and a "Message déchiffré" output field with the text "salut 2026". The interface also features a "Cipher Feedback - Mode flot" label in the top right corner.

FIGURE 7.6 – Chiffrement et déchiffrement DES en mode CFB

El Ghamal

ElGamal

Asymétrique - Logarithme discret

Générer les clés ElGamal

Reset

Clé publique:

$p=66141290155392086956798031962833010717124734308291090502880953129751396348183$, $g=5$,
 $y=27902947302926811721931356487253536145995464452054156346045867178247781295335$

Clé privée: $x=1982663629429871756631278923159102281933186402395961610724985217603599959986$

CHIFFREMENT

MESSAGE

salut 19

P

$661412901553920869567980319628330107171247343$

G

5

Y

$279029473029268117219313564872535361459954644$

Chiffrer

Reset

Résultat chiffré (a:b,...)

DÉCHIFFREMENT

TEXTE CHIFFRÉ

$021/013488,5305/284//01369601846935305/80/9/5$
 $0342570079982415312056604842157250961886376:3$
 $465798455245948042934699150714024692007274821$
 $5173698984775268729875856145919$

X (PRIVÉ)

$198266362942987175663127892315910228193318640$

P

$661412901553920869567980319628330107171247343$

Déchiffrer

Reset

Message déchiffré

salut 19

FIGURE 7.7 – El Ghamal

EC El Ghamal

Générer les clés EC-ElGamal

Reset

Clé publique Q:

(63253785385836706378567962226561856563753296132861956437900303085255243613463,
76597037091435996408101457989983885435666844989629570222037243803171836799630)

Clé privée k: 8378825018397636165259017514485605142331066752137287923612917200693633431503

CHIFFREMENT

MESSAGE

salut 2026

QX

632537853858367063785679622265618565637532961

QY

765970370914359964081014579899838854356668449

Chiffrer

Reset

Résultat chiffré (points)

4847749302721433347926126768928238337901662416707015
0175072833182335470858845:71494272062909134201043543
980522732595702402520598026523133115857735540434346:
2538900272750102112978750213326184347191757694895669
7748854846600526432196889:10050781738171908149633882
539428890857837786085025033619389210188786303898326|
7569193829372573688976813812794218786929303901145539
3798879421933762975130779:22994631017963399872871792

DÉCHIFFREMENT

TEXTE CHIFFRÉ

18104553340596/0815918426/19210436/592193/095
455934421811809230932394:38722386253785498364
11037776900092062932481196525148508499204112
814871873847

K (PRIVÉ)

837882501839763616525901751448560514233106675

Déchiffrer

Reset

Message déchiffré

salut 2026

FIGURE 7.8 – EC El Ghamal

Chapitre 8

Conclusion et Perspectives

8.1 Bilan du projet

Ce projet nous a poussés bien au-delà de ce qu’une implémentation académique exige habituellement. Coder DES depuis ses tables de permutation, c’est comprendre physiquement pourquoi les S-Boxes sont indispensables à la non-linéarité. Implémenter MixColumns dans $GF(2^8)$, c’est toucher du doigt la théorie des corps finis. Dérouler le double-and-add sur secp256k1, c’est réaliser pourquoi les courbes elliptiques ont révolutionné la cryptographie mobile.

Indicateur	Valeur
Lignes de code totales	$\approx 5\,500$
Algorithmes implémentés	10 (+ 3 modes d’opération)
Bibliothèques cryptographiques externes	0
Templates HTML	4
Routes Flask	12

8.2 Perspectives d’amélioration

Plusieurs pistes auraient enrichi le projet si le temps l’avait permis :

- **Chiffrement de fichiers** — Uploader une image et la chiffrer directement avec AES, en visualisant les différences entre les modes ECB (motifs visibles) et CBC (résultat uniforme).
- **Outil de** — Un bouton « Casser » pour César (26 essais en clair) et une analyse fréquentielle interactive pour Vigenère.
- **Authentification à deux facteurs (2FA)** — Via TOTP et une application mobile, pour illustrer concrètement comment les systèmes d’authentification modernes se construisent.
- **Génération d’IV avec `os.urandom()`** — Remplacer le générateur `random` par une source d’entropie cryptographiquement sûre, ce qui serait impératif en contexte de production.

8.3 Réflexion finale

La cryptographie est une discipline rare où les mathématiques les plus abstraites rencontrent les enjeux de société les plus concrets. Vie privée, sécurité nationale, liberté d'expression — tout cela repose, au fond, sur des équations, des tables de permutation et des multiplications dans des corps finis.

Ce qui frappe le plus après ce projet, c'est la fragilité de chaque algorithme face au temps. DES était inviolable en 1977. Il était cassé en 1999. AES est le standard incontesté aujourd'hui. Dans vingt ans, avec l'ordinateur quantique, qui sait ? C'est peut-être la leçon la plus profonde : *en cryptographie, rien n'est éternel*. Sauf, peut-être, la beauté des mathématiques qui la fondent.

« La seule manière de savoir si un système est sécurisé est de tenter de le casser. Et même quand on n'y arrive pas, on n'est jamais tout à fait sûr. »

— Bruce Schneier, cryptographe

Chapitre A

Tableau Récapitulatif des Algorithmes

Algorithme	Année	Type	Clé	État actuel
César	−50 av. J.-C.	Subst. mono.	Entier 1–25	Cassé
Vigenère	1553	Subst. poly.	Mot (lettres)	Cassé (1863)
Vernam OTP	1917	Flot (XOR)	= longueur message	Parfait
RC4	1987	Flot (KSA)	Variable	Interdit TLS
DES (ECB)	1977	Blocs 64 bits	56 bits	Cassé (1999)
DES-CBC	1980	Blocs chaî- nés	56 bits + IV	Obsolète
DES-CFB	1980	Flot sur DES	56 bits + IV	Obsolète
AES-128	2001	Blocs 128 bits	128 bits	Standard
RSA	1977	Asymétrique	512–4096 bits	Sûr (≥ 2048)
ElGamal	1985	Asymétrique	128+ bits	Sûr
EC-ElGamal	1985	Asym. ECC	256 bits	Sûr
SHA-256	2001	Hachage	256 bits	Standard

TABLE A.1: Récapitulatif complet des algorithmes implémentés dans CryptoApp

Chapitre B

Références Bibliographiques

1. Shannon, C. E. (1949). *Communication Theory of Secrecy Systems*. Bell System Technical Journal.
2. Diffie, W., & Hellman, M. (1976). *New Directions in Cryptography*. IEEE Transactions on Information Theory.
3. Rivest, R., Shamir, A., & Adleman, L. (1978). *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*. Communications of the ACM.
4. NIST (2001). *Advanced Encryption Standard (AES)*. FIPS PUB 197.
5. Daemen, J., & Rijmen, V. (2002). *The Design of Rijndael : AES — The Advanced Encryption Standard*. Springer.
6. Koblitz, N. (1987). *Elliptic Curve Cryptosystems*. Mathematics of Computation, 48(177).
7. ElGamal, T. (1985). *A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms*. IEEE Transactions on Information Theory.
8. RFC 7465 (2015). *Prohibiting RC4 Cipher Suites*. IETF.